

Ghazi Muhammad Abdullah

Domain: Artificial Intelligence & Machine Learning

Submitted to : Mr. Mohazin Zahid

Experiment 2

Lab Tasks



Experiment 2.py

Viva Questions

1. Type casting means converting a value from one data type to another data type. It is needed after `input()` because the `input()` function always returns data as a string, which is text. If we want to perform mathematical operations like addition, subtraction, or multiplication on the user's input, we first need to convert that string into a number using `int()` for whole numbers or `float()` for decimal numbers. For example, if we write `age = input("Enter age: ")`, the variable `age` will store the value as text like "25". If we try to do `age + 1`, Python will give an error because we cannot add a number to text. But if we write `age = int(input("Enter age: "))`, now `age` becomes the number 25 and we can easily do `age + 1` to get 26. So type casting is essential whenever we need to perform calculations on user input.
2. The main difference between a list and a tuple is that a list is mutable, meaning we can change, add, or remove items from it, while a tuple is immutable, meaning once created, we cannot change it. Lists use square brackets like `fruits = ["apple", "banana"]` and we can add new items using `fruits.append("orange")` or change existing items like `fruits[0] = "mango"`. Tuples use parentheses like `colors = ("red", "green",`

"blue") and we cannot add, remove, or change any items after creation. If we try to do `colors.append("yellow")`, Python will give an error. Because tuples cannot be changed, they are faster and use less memory than lists. We should use lists when our data might change, like a shopping cart or student list, and use tuples when our data is fixed and should never change, like the days of the week, coordinates of a location, or a person's date of birth.

3. A for loop is used when we know exactly how many times we want to repeat a block of code, while a while loop is used when we want to repeat code until a certain condition becomes false and we do not know in advance how many times it will repeat. For example, if we want to print numbers from 1 to 10, we know it will run 10 times, so we use a for loop: `for i in range(1, 11): print(i)`. This is safe and clean because we know exactly how many iterations will happen. On the other hand, if we want to keep asking the user to enter numbers until they enter 0, we do not know how many numbers they will enter, so we use a while loop: `num = 1; while num != 0: num = int(input("Enter number: "))`. This loop will keep running until the user types 0. The main risk with while loops is that if we forget to update the condition, the loop can run forever and crash our program, which is called an infinite loop. So we use for loops when the count is known and while loops when we need to repeat until a condition changes.
4. The return keyword does three important things inside a function. First, it sends a value back to the place where the function was called, so we can use that value later. Second, it immediately stops the function from running any further, meaning any code after the return statement will not be executed. Third, if we do not write a return statement, the function automatically returns None, which means nothing. For example, if we write `def add(a, b): return a + b` and then call `result = add(5, 3)`, the function will return 8 and store it in the variable result. The return statement gives us the ability to get results from functions and use them elsewhere in our program. If a function only prints something but does not return a value, like `def say_hello(): print("Hello")`, and we write `result = say_hello()`, then result will be None because there is no return. So return is essential when we want our function to produce a value that we can use later in our code.

5. Dictionaries are extremely useful for storing data because they use key-value pairs, which means we can access data using a meaningful name instead of a number. This makes our code much more readable and easier to understand. In a list, we access items by their position, like `students[0]` which is confusing because we don't know what 0 represents. But in a dictionary, we use keys like names, such as `student["name"]` or `student["age"]`, which clearly tells us what data we are accessing. Dictionaries also make it very fast to look up, add, or change values. A real world example is storing student records. Instead of using multiple lists for names, ages, and grades, we can use a single dictionary for each student like `student1 = {"name": "Ali", "age": 20, "grade": "A"}` and `student2 = {"name": "Sara", "age": 22, "grade": "B"}`. Then we can easily access `student1["name"]` to get Ali, or `student2["grade"]` to get B. If we have many students, we can store all of them in a list of dictionaries and easily search, update, or display any student's information. This is much more organized and efficient than using separate lists for each attribute.
6. The equals sign `=` and the double equals sign `==` have completely different purposes in Python. The single equals sign `=` is called the assignment operator and it is used to store a value in a variable. For example, `name = "John"` means we are assigning the text "John" to the variable called name. The double equals sign `==` is called the comparison operator and it is used to check if two values are equal to each other. It returns True if they are equal and False if they are not. For example, `if name == "John"` means we are checking whether the variable name contains the value "John". We use `=` when we want to give a value to a variable, and we use `==` when we want to compare two values, especially inside if statements and loops. A common mistake beginners make is using `=` instead of `==` in conditions, like writing `if x = 5` instead of `if x == 5`, which will cause an error. So always remember: `=` is for assignment (giving a value), and `==` is for comparison (checking if values are equal).

MCQ's

- Q1. What is the index of the first character of a string?
(b) 0

Q2. Which loop is best when the number of repetitions is known?

(b) for

Q3. What does len("hello") return?

(b) 5

Q4. Which keyword sends a value back from a function?

(c) return

Q5. A collection of unique values is a:

(c) set

Intern Name: Ghazi Muhammad Abdullah

Supervisor Name: Mr. Mohazin Zahid

Date: 06 July, 2026

Domain: AI/ML Intern

Intern Signature:

A handwritten signature in black ink, appearing to read 'Ghazi', with a stylized flourish underneath.